

講義 13：第一階段總整理——完整報表架構與實作攻略（授課順序：接在講義 14 之後）

對應練習：ex13（第一階段總整理實作） | 答案程式：ZR_TR13_CAPSTONE

本講重點

- 把前面各講（ALV 除外）的技能拼成一支正式等級的傳統報表
- 完整報表的標準架構與撰寫順序
- 唯一的新技巧：總頁數回填（READ LINE / MODIFY LINE），延伸到一份清單多張文件各自計算頁次
- 實作攻略與自我檢查清單
- 第一階段結業對照：讀懂正式程式 Z_INVENTORY_COST_REPORT，以及實戰案例「傳票清單 ZRFI0004」
- 用前面所學，從需求開始完成一支好的傳票清單，並用正式程式驗證

1. 總整理目標

做一張「航班營收報表」：選擇畫面過濾 → JOIN 取數 → 計算營收 → 132 欄分頁排版 → 頁首頁尾 → 頁次「n / 總頁數」。這正是實務傳統報表的完整形狀——完成它，就具備獨立接報表需求的能力。

2. 完整報表架構（技能總地圖）

```
REPORT zr_tr13_capstone NO STANDARD PAGE HEADING
                                LINE-SIZE 132
                                LINE-COUNT 65(3).           " ← 講義 12
TABLES sflight.                                               " ← SELECT-OPTIONS 參考欄位用

TYPES: BEGIN OF ty_rev, ... END OF ty_rev.                 " ← 講義 3：自訂結果結構
DATA: gt_rev TYPE STANDARD TABLE OF ty_rev, ...           " ← 講義 4

SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE t_b1.   " ← 講義 7
  SELECT-OPTIONS: s_carrid FOR sflight-carrid,
                  s_fldate FOR sflight-fldate.
  PARAMETERS p_zero AS CHECKBOX DEFAULT 'X'.
SELECTION-SCREEN END OF BLOCK b1.

INITIALIZATION.                                              " ← 講義 10：事件
  t_b1 = '查詢條件'.

START-OF-SELECTION.
  PERFORM get_data.                                          " ← 講義 8：事件只放 PERFORM
  PERFORM display_data.

END-OF-SELECTION.
  PERFORM update_total_pages.                               " ← 本講新技巧
```

TOP-OF-PAGE.	" ← 講義 12：頁首 + 欄位標題
END-OF-PAGE.	" ← 講義 12：頁尾
* FORM 區集中檔尾 (正式版再依講義 14 拆 _TOP/_F01)	

各 FORM 用到的技能：

FORM	內容	用到的講義
<code>get_data</code>	INNER JOIN SFLIGHT + SCARR、IN 過濾、DELETE WHERE、LOOP 算營收 + MODIFY	11、7、5
<code>display_data</code>	空表防呆、固定欄位座標 WRITE、CURRENCY、合計筆數	4、12
<code>update_total_pages</code>	總頁數回填	本講

3. 新技巧：總頁數回填

頁首要印「頁次 3 / 12」，但印第 3 頁時總頁數還不知道。解法靠清單的特性：**WRITE 不是直接上螢幕**，而是先寫進清單緩衝區 (**List Buffer**) (講義 10 第 3.5 節)；END-OF-SELECTION 時全部頁面都已生成，**sy-pagno** 就是總頁數——回頭把每頁頁首的佔位字串改掉即可：

```
* 頁首先印佔位符 ( TOP-OF-PAGE 內 ) :
WRITE: ... '頁次      : ', (3) sy-pagno NO-GAP, '/', '###'.

* 全部輸出完 ( END-OF-SELECTION ) 再回填 :
FORM update_total_pages.
  DATA lv_total TYPE c LENGTH 3.
  lv_total = sy-pagno.                " 此刻的頁次 = 總頁數

  DO sy-pagno TIMES.
    READ LINE 2 OF PAGE sy-index.      " 讀回第 n 頁第 2 行 → sy-lisel
    IF sy-subrc = 0.
      REPLACE '###' WITH lv_total INTO sy-lisel.
      MODIFY LINE 2 OF PAGE sy-index.  " 改完寫回 buffer
    ENDIF.
  ENDDO.
ENDFORM.
```

- **READ LINE n OF PAGE p**：把清單第 p 頁第 n 行的內容讀進 **sy-lisel**。
- **MODIFY LINE**：把改好的 **sy-lisel** 寫回同一行。
- 佔位符 (**###**) 要選內容裡不會自然出現的字串，行號要跟頁首版型一致 (範例中「頁次」在頁首第 2 行)。
- **MODIFY CURRENT LINE**：改寫「最近一次 **READ LINE** 讀到的那一行」，效果等於 **MODIFY LINE n OF PAGE p**，舊程式常見 (第 6 節案例就是這樣寫)。

3.1 延伸：一份清單印多張文件，每張各自「頁次 n / 總頁數」

傳票、出貨單這類報表，一次會印好幾張文件，每張都要從第 1 頁算起。**sy-pagno** 是**整份清單**的頁數，一路累加，不能直接拿來當某張文件的總頁數。做法是**記下每張文件從第幾頁開始、到第幾頁結束**，最後再逐張回填：

```
TYPES: BEGIN OF ty_docpage,
        doc          TYPE s_carr_id,      " 實務上換成文件鍵值，如 BUKRS/BELNR/GJAHR
        first_page TYPE i,
        last_page  TYPE i,
    END OF ty_docpage.
DATA: gt_docpage TYPE STANDARD TABLE OF ty_docpage,
      gs_docpage TYPE ty_docpage.
```

TOP-OF-PAGE.

```
WRITE: /1 '文件：', gv_doc, 60 '頁次：', (9) '###/###'.
ULINE.
```

* 輸出 (END-OF-SELECTION 內)：每張文件換新頁，記下起訖頁

```
LOOP AT gt_flight INTO gs_flight.
    AT NEW carrid.
        gv_doc = gs_flight-carrid.
        NEW-PAGE.                " 每張文件從新的一頁開始
        CLEAR gs_docpage.
        gs_docpage-doc = gs_flight-carrid.
    ENDAT.

    WRITE: / gs_flight-connid, gs_flight-fldate.
    IF gs_docpage-first_page = 0.
        gs_docpage-first_page = sy-pagno.    " 第一行寫出後，sy-pagno 就是這張的第一頁
    ENDIF.

    AT END OF carrid.
        gs_docpage-last_page = sy-pagno.    " 最後一行寫出後，就是這張的最後一頁
        APPEND gs_docpage TO gt_docpage.
    ENDAT.
ENDLOOP.
```

* 回填：逐張文件、逐頁把佔位符換成「第幾頁/總頁數」

```
LOOP AT gt_docpage INTO gs_docpage.
    lv_total = gs_docpage-last_page - gs_docpage-first_page + 1.
    lv_page  = gs_docpage-first_page.
    WHILE lv_page <= gs_docpage-last_page.
        lv_n  = lv_page - gs_docpage-first_page + 1.
        lv_nc = lv_n.
        lv_tc = lv_total.
        CONDENSE: lv_nc, lv_tc.
        CONCATENATE lv_nc '/' lv_tc INTO lv_text.
```

```

READ LINE 1 OF PAGE lv_page.          " 頁次在頁首第 1 行
IF sy-subrc = 0.
    REPLACE '###/###' WITH lv_text INTO sy-lisel.
    MODIFY LINE 1 OF PAGE lv_page.
ENDIF.
lv_page = lv_page + 1.
ENDWHILE.
ENDLOOP.

```

實測 (`ZR_TR13_DOCPAGE_TEST` , 2026-09-29 , `LINE-COUNT 10` 讓每張都跨頁) : AA 印出 1/4 ~ 4/4 、 LH 印出 1/10 ~ 10/10 , UA 又從 1/8 開始。只有一頁的文件 , 起訖頁相同 , 自然算出 1/1 。

注意事項：

- 每張文件都要 `NEW-PAGE` , 否則兩張文件擠在同一頁 , 起訖頁會重疊。
- `READ LINE ... OF PAGE p` 的 `p` 是整份清單的實際頁序 , 不受程式怎麼顯示頁碼影響 (第 6 節案例會看到這點) 。
- 頁尾若是在明細之後自己印 (講義 12 第 3 節) , 要在記錄 `last_page` 之前印完 , 不然頁尾被擠到下一頁時 , 那一頁不會算進這張文件。

4. 實作攻略 (建議順序)

1. 先讓資料對：宣告 + 選擇畫面 + `get_data` , 用最陽春的 `LOOP + WRITE` 驗證 JOIN 結果與營收計算正確。資料錯 , 排版再漂亮都是白工。
2. 再排版：畫欄位座標表 (欄名 / 起始欄 / 寬度) , 完成明細行與 TOP-OF-PAGE 欄位標題 , 確認直欄對齊。
3. 再分頁：加 `LINE-COUNT` 與 `END-OF-PAGE` , 用足量測試資料驗證跨頁行為。
4. 最後回填總頁數：確認頁首版型固定後才做 (行號寫死在 `READ LINE` 裡 , 版型再改要同步) 。
5. 全程遵守鐵律：每個 `SELECT / READ TABLE / CALL FUNCTION` 之後檢查 `sy-subrc` ; 查無資料要有友善訊息。

5. 自我檢查清單 (第一階段結業標準)

- ☐ 選擇畫面有 BLOCK 框與標題 , 條件全部生效 (含空條件 = 查全部)
- ☐ JOIN 一次取數 , 沒有迴圈內 SELECT
- ☐ 營收計算正確 (票價 × 已售座位 , 金額欄含幣別格式)
- ☐ 每頁頁首 (含頁次) 、頁尾齊全 , 直欄筆直 , 跨頁正常
- ☐ 總頁數回填正確 (每一頁都顯示相同總頁數)
- ☐ 主流程只有 PERFORM , 邏輯都在 FORM ; 命名符合 `gv_/gs_/gt_` 慣例
- ☐ 查無資料、輸入錯誤都有明確訊息 , 不會 dump

6. 第一階段結業對照：讀正式程式

打開 `src/` 的 `Z_INVENTORY_COST_REPORT` (或 `ZDQM` 系列) 對照閱讀：結構跟你的總整理作品一模一樣——宣告 (或 `_TOP include`) 、選擇畫面、事件骨架、FORM 區、分頁排版。差別只在業務邏輯的複雜度。讀得懂、指得出每一段對應哪一講 , 本課程即結業；下一階段是 OOP 課程 (`src/ABAP_Training_OOP/`) , 把同樣的邏輯改用 Class 組織。

6.1 實戰閱讀：傳票清單 ZRFI0004

完整原始碼：[cases/zrfi0004.prog.abap](#)。這是一支真實的客戶程式，2006 年寫成，之後經過多人多次修改，註記和被註解掉的舊程式碼都原樣保留。它依賴客戶自建的表 **ZFI0037** 與 T-code，課堂系統無法執行，**只供閱讀**。

功能：選一個公司代碼和條件，把符合的會計傳票（FI 文件）一張張印成紙本傳票，每張傳票底部有會計主管 / 覆核 / 製票員簽核欄、本頁合計、累頁合計，以及「頁碼 n/總頁數」。

閱讀前的業務背景（FI 會計模組）：

表	內容
BKPF	會計文件表頭（公司代碼 BUKRS + 文件號碼 BELNR + 會計年度 GJAHR 是鍵值）
BSEG	會計文件明細（借貸方、科目、金額）
VBSEGA / VBSEGS / VBSEGD / VBSE GK	暫存（Park）文件的明細，依科目類型分四張：資產 / 總帳 / 客戶 / 供應商
BKPF-BSTAT	文件狀態：空白 = 已過帳、 V = 暫存、 Z = 被刪除的暫存文件
USR21 → ADRP	由 SAP 帳號查使用者的姓名

程式架構與對應講義

區塊	內容	講義
REPORT ... LINE-SIZE 203 LINE-COUNT 65(0)	寬報表、不保留頁尾（原因見下方分頁設計）	12
TABLES、LIKE、BEGIN OF ... OCCURS 0、OCCURS 0 WITH HEADER LINE	舊式宣告，全部內表都帶 Header Line	2、5 58、7
DEFINE cls	macro，一次做 CLEAR 加 REFRESH	9（後面才教，先看懂）
選擇畫面：BLOCK、OBLIGATORY、DEFAULT、AS CHECKBOX、MATCHCODE OBJECT、NO-DISPLAY		7
INITIALIZATION 裡的 SELECTION-SCREEN FUNCTION KEY、AT SELECTION-SCREEN 的 CALL TRANSACTION	選擇畫面上的兩顆工具按鈕，跳到 SM30 維護 / 查詢會計主管表 ZFI0037 （見下方說明）	28（選修，先看懂）
check_auth_object：AUTHORITY-CHECK 依序試 ACTVT 01 / 02 / 03	有新增、修改、顯示任一權限就放行	28

區塊	內容	講義
<code>initial_document_status_range</code> ：呼叫 <code>GET_DOMAIN_VALUES</code> 取 Domain 固定值，在程式裡自己填 NO-DISPLAY 的 <code>s_bstat</code>	把三個勾選框轉成 range 表，再用 IN 查詢	7、15、25
<code>extract_acct_doc_header_data</code> ：SELECT ... IN、查無資料 MESSAGE ... STOP		6、7、10
同一個 FORM 後半：SELECT ... ENDSELECT 加 ORDER BY ... DESCENDING、抓到第一筆就 EXIT	找「生效日不晚於文件日期的最近一任會計主管」	6 §4
<code>extract_acct_doc_item_data</code> ：FOR ALL ENTRIES、APPENDING CORRESPONDING FIELDS、欄位別名 <code>saknr AS hkont</code> 、MOVE-CORRESPONDING	過帳文件讀 BSEG；暫存文件讀四張 VBSEG* 後合併	3、11
<code>extract_acct_text_data</code> ：LOOP 裡逐筆 SELECT SINGLE 查科目、成本中心、客戶 / 供應商名稱	可以看懂，但是效能反面教材	11
<code>write_report</code> ：AT NEW <code>gjahr</code> 後馬上 READ TABLE <code>t_dochd</code> INDEX <code>l_tabix</code>	繞過 AT 區塊把右邊欄位遮成 * 的規則	20
WRITE AT (<code>t_optfm-hkont</code>) ...	欄寬集中寫在結構 <code>t_optfm</code> ，就是「欄位座標表」的程式版	12
<code>split_sgtxt</code> ：用 <code>cl_abap_list_utilities=>dynamic_output_length</code> 逐字算顯示寬度，把內文切成兩行	中文字佔兩格	12 §2.1、18
<code>write_header</code> ：convert_string_to_xstring 算字串位元組數、用 <code>sy-linsz</code> 算置中位置	中文標題置中	12

選擇畫面上的兩顆按鈕（講義 28 第 7 節才正式教）

執行畫面最上方，標題「傳票清單」下面那一列的「維護會計主管名稱」「顯示會計主管名稱」，是選擇畫面工具列上的按鈕，分三步加進來：

```
" 1. 宣告：TABLES sscrfields 是選擇畫面的系統結構；smp_dyntxt 放「圖示+文字」
TABLES sscrfields.
DATA functxt TYPE smp_dyntxt.
```

```
" 2. 在選擇畫面宣告區加按鈕，最多 4 顆（FUNCTION KEY 1~4）
SELECTION-SCREEN: FUNCTION KEY 1,
                  FUNCTION KEY 2.
```

```
INITIALIZATION.
```

```
" 3. 設定按鈕的圖示和文字：functxt_01 對應 FUNCTION KEY 1，以此類推
functxt-icon_id   = icon_tools.      " 圖示（扳手工具）
```

```

functxt-icon_text = text-t03.      " 文字：維護會計主管名稱
sscrfields-functxt_01 = functxt.

CLEAR functxt.                    " 同一個結構重複用，先清空
functxt-icon_id   = icon_tools.
functxt-icon_text = text-t04.      " 文字：顯示會計主管名稱
sscrfields-functxt_02 = functxt.

```

- 只要文字、不要圖示時，直接寫 `sscrfields-functxt_01 = '維護會計主管名稱'`，就好。
- 要圖示 + 文字時，才用 `smp_dyntxt` 結構：`icon_id` 放圖示常數，`icon_text` 放文字。
- **icon_tools 是什麼**：SAP 內建的圖示常數，代表「工具（扳手）」圖示，值是 '@45@'。畫面看到 @代碼@ 這種格式，就會顯示成對應的圖示。這些常數定義在 Type Group **ICON**，原程式第 29 行的 **TYPE-POOLS: icon**。就是載入它；新版系統會自動載入，不寫也能用。程式裡用常數名稱，不要直接寫 '@45@'，比較看得懂。
- 要換別的圖示，SE38 執行報表 **SHOWICON** 可以看到全部圖示和常數名稱。常用圖示對照與其他用法（清單、ALV 裡放圖示）見講義 28 第 7.1 節。
- 原程式把 **SELECTION-SCREEN FUNCTION KEY** 寫在 **INITIALIZATION** 事件裡面。它是宣告，不是執行時才跑的指令，寫在哪裡都會生效，但容易讓人誤以為它是事件裡的邏輯。自己寫時放在 **PARAMETERS / SELECT-OPTIONS** 那一區比較清楚；**INITIALIZATION** 裡只留設定按鈕文字的程式。

按下按鈕後的處理：

```

AT SELECTION-SCREEN.
CASE sscrfields-ucomm.
  WHEN 'FC01'.
    CALL TRANSACTION 'ZFI0037'.    " 按鈕 1
  WHEN 'FC02'.
    CALL TRANSACTION 'ZFI0037Q'.   " 按鈕 2
ENDCASE.

```

- 按下按鈕會觸發 **AT SELECTION-SCREEN**，`sscrfields-ucomm` 分別是 'FC01'、'FC02'（系統固定的代碼，不能自己改）。
- **CALL TRANSACTION 'ZFI0037'** 跟在命令欄輸入 **ZFI0037** 一樣，會開啟這個 T-code 的畫面。
- **ZFI0037、ZFI0037Q 不是程式，是 SE93 建的 Parameter Transaction**。這種 T-code 不指向自己的程式，而是呼叫另一個 T-code（這裡是 SM30），並預先填好那個畫面的欄位：
 - SM30 的「Table/View」欄位先填好要維護的對象：**ZFI0037** 填會計主管表 **ZFI0037** 本身（簽核欄的主管姓名就從這張表查）；**ZFI0037Q** 填的是另一個唯讀 View（見下方）。
 - 勾選「Skip initial screen」，跳過 SM30 的初始畫面，直接進入維護畫面。
 - **ZFI0037Q** 的結尾 **Q** 是查詢（Query）版，只能看、不能改。

所以按鈕的用途是：印傳票前，發現主管資料不對，可以直接按按鈕進 SM30 修改，不用另外記 T-code；只負責查詢的人用第二顆按鈕。

查詢版為什麼要另外建一個唯讀的 **Maintenance View**

SM30 維護畫面的工具列有一顆「Display ↔ Change」切換按鈕。就算 Parameter Transaction 讓 SM30 以顯示模式開啟，使用者只要有這張表的維護權限，按一下切換按鈕就能改資料。只在 T-code 上設定「顯示」，擋不住這一步。

所以查詢版不讓 SM30 直接開 ZFI0037 這張表，而是在 SE11 另外建一個 **Maintenance View**，在 **Maint. Status** 頁籤把 Access 設成 **Read only**，再讓 ZFI0037Q 開這個 View：

T-code	SM30 開啟的對象	結果
ZFI0037	表 ZFI0037	可以新增、修改
ZFI0037Q	唯讀的 Maintenance View	只能顯示，畫面上沒有 Change / Display 切換

控制放在 View 的定義上，不管使用者怎麼進 SM30、有什麼權限，這個 View 都改不了資料。

現在只要看得懂這段程式在做什麼。Maintenance View 在講義 21 介紹；SE93 建立 Parameter Transaction 的步驟、要填哪些欄位，在講義 28 第 7 節。講義 28 也會說明這種做法的缺點：Parameter Transaction 只是跳進 SM30，本身沒有業務權限檢查、沒有鎖定，實務上要另外包一層檢查程式。

分頁設計：自己控制整頁 65 行

這支程式完全不用 **END-OF-PAGE**，每一頁的每一行印在哪裡都是自己決定的：

行號	內容	程式
1 ~ 6	空白 (連續報表紙上方留白)	write_header 開頭 SKIP TO LINE 7
7 ~ 10	公司名稱、空行、「年度 + 傳票清單」、橫線	write_header
11 ~ 14	文件日期 / 幣別等表頭資訊、===、欄位標題、===	report_title
15 ~ 54	明細，每筆 4 行 (第一行內容、SKIP 空一行、第二行內容、虛線)，每頁最多 10 筆，10 筆剛好排滿	write_report 的 LOOP AT t_docit
(明細不滿 10 筆)	剩下的行留白，簽核區照樣從第 55 行開始	
55 ~ 59	橫線、簽核欄 + 本頁合計、只有一條直線的分隔行、累頁合計、橫線	SKIP TO LINE 55 之後
60 ~ 65	過帳日期、會計文件、第 64 行「頁碼 n/&」、結尾虛線	report_footer

為什麼每 10 筆換頁：l_modno = l_pgcnt MOD 10，等於 0 就設換頁註記 w_endfg = 'P'；整張傳票的最後一筆則在 AT END OF gjahr 設 w_endfg = 'L'。兩種註記都會跳到第 55 行印簽核區和頁尾，一路印到第 65 行。下一筆明細的 WRITE / 超出頁面，系統自動換頁並觸發 TOP-OF-PAGE。

為什麼是 65(0)：頁尾的 6 行 (60 ~ 65) 是由 write_report 自己印的。如果寫 65(6)，第 60 ~ 65 行被系統保留給 END-OF-PAGE，程式自己印到第 60 行時就會觸發換頁，頁尾被擠到下一頁。2023/10/19 那次修改就是把頁尾從 END-OF-PAGE 搬進 write_report，同時把 65(6) 改成 65(0)。

每張傳票的「頁碼 n/總頁數」

1. **n** : `report_footer` 用 `sy-pagno` 組出 `n/&` , `&` 是總頁數的佔位符。
2. 每張從 **1** 算起 : `AT NEW gjahr` 裡先 `NEW-PAGE` , 再直接寫 `sy-pagno = 1` 。實測 (2026-09-29) 這樣改之後 , 頁首頁尾看到的 `sy-pagno` 確實會從 1 重新算 , 下一頁變 2 。
3. 總頁數 : 程式不看 `sy-pagno` , 改用自己算的 `l_doc_page` (這張幾頁) 和 `l_total_page` (整份清單累計幾頁) , 每滿 10 筆各加 1 。
4. 回填 : 一張傳票印完 , 從 `l_total_page - l_doc_page + 1` 這一頁開始 , 逐頁 `READ LINE 64 OF PAGE` → 把 `&` 換成總頁數 → `MODIFY CURRENT LINE` 。

第 3 點是關鍵 : `sy-pagno` 被改回 **1** 之後 , 就不再等於清單的實際頁序 , 但 `READ LINE ... OF PAGE` 要的是實際頁序 (實測 : 第二張文件的第 1 頁 , 用 `OF PAGE 3` 才讀得到) 。所以程式必須另外用 `l_total_page` 記錄實際頁序。這也是為什麼**不建議寫入系統欄位** : 寫了之後 , 系統欄位和系統內部的狀態就對不上 , 後面的程式要一直記得這件事。第 3.1 節「記下每張文件的起訖頁」的做法不必改 `sy-pagno` , 比較乾淨。

另一個細節 : 傳票剛好 10、20、30 筆時 , 最後一筆會同時觸發「滿 10 筆多算一頁」和「傳票結束」。
2023/11/02 的修改在 `AT END OF gjahr` 裡把多算的那一頁扣回來。這類邊界條件 , 是自己計算頁數時最容易出錯的地方。

值得討論的寫法 (看得懂 , 自己寫時換個做法)

寫法	問題	建議
<code>sy-pagno = 1.</code>	寫入系統欄位 , 之後 <code>sy-pagno</code> 跟實際頁序對不上	用第 3.1 節的起訖頁做法
<code>w_subrc = sy-subrc.</code> 放在回填迴圈之後 , 用來判斷「這張傳票有沒有明細」	這時的 <code>sy-subrc</code> 來自迴圈裡最後一次 <code>REPLACE</code> , 跟有沒有明細只是間接相關	<code>sy-subrc</code> 要在產生它的陳述式後馬上檢查 (講義 4) , 有沒有明細應該直接判斷
<code>LOOP</code> 裡逐筆 <code>SELECT SINGLE</code> (科目、成本中心、客戶、供應商)	明細 1,000 筆就查資料庫好幾千次	先整批讀進內表再 <code>READ TABLE</code> , 或用 <code>JOIN</code> (講義 11)
全部內表都帶 Header Line	同一個名字同時代表整張表和一列	<code>TYPE STANDARD TABLE OF</code> 加獨立 work area (講義 5 §8)
簽核欄的三段 <code>IF</code> 分支大部分程式碼重複	改一個地方要改三次	抽成 FORM , 差異用參數傳入 (講義 8)
大量被註解掉的舊程式碼	越來越難讀	舊版本交給版本管理 (講義 8a 的 TR 版本) , 程式裡只留修改註記

6.2 用前面所學 , 完成一支好的傳票清單

6.1 是「讀懂別人的程式」。這一節換個角度 : 如果這個需求交給你 , 用第一階段學過的寫法 , 從頭寫出一支**正確、好維護的傳票清單**。步驟照第 4 節的實作攻略 : 先弄清需求、先讓資料對、再排版、再分頁、最後回填頁數。

完整範例程式 : [cases/zr_tr13_zrfi0004.prog.abap](#) (課堂系統 `$TMP` 的 `ZR_TR13_ZRFI0004` , 可以直接執行) 。

步驟 1 : 把需求寫清楚

動手前先把需求條列出來，這一步做好，後面每個 FORM 要做什麼就很清楚：

項目	需求
條件	公司代碼 (必填) 、文件號碼、年度、文件類型、過帳日、文件日、輸入日、過帳者、暫存者；三個勾選框選文件狀態：過帳文件 / 暫存文件 / 被刪除的暫存文件
輸出單位	一張傳票從新的一頁開始，每頁最多 10 筆明細
每筆明細	科目、對象代號、成本中心、利潤中心、內部訂單、指派、參考碼三、借方或貸方金額、內文；第二行印各代號的名稱，內文太長接到第二行
對象代號	資產科目放資產號碼；客戶 / 供應商放代號，名稱欄放搜尋名稱
每頁底部	會計主管 / 覆核 / 製票員簽核欄、本頁合計、累頁合計、過帳日期、文件號碼、「頁碼 n/總頁數」(每張傳票各自計算)
簽核欄	會計主管依「傳票日期當時」的主管 (ZFI0037 依生效日維護) ；暫存文件只印主管與製票員；已過帳文件三者都印並加日期
沒有明細的傳票	只印表頭資訊與簽核欄

整理需求時就發現一個問題：選擇畫面有「文件日期」條件，但原程式的 SELECT 根本沒用它，使用者填了也沒效果。這種事要跟使用者確認，不能自己決定。範例程式為了能跟原程式逐行比對，保留一樣的行為，在程式裡註明；確認後只要在 WHERE 加一行 `AND bldat IN s_bldat`。

步驟 2：規劃版面 (講義 12)

先畫出整頁 65 行的配置 (第 6.1 節的行號表)，再決定：

- `LINE-SIZE 203`、`LINE-COUNT 65(0)`：頁尾要固定印在第 55~65 行，而且每頁都要有，所以不用 `END-OF-PAGE`，自己印 (原因見 6.1)。列印時選 `X_65_255` (講義 12 §1.1)。
- 每欄寬度集中在一個結構，頁首、明細、合計列都照它寫，要調欄寬只改一個地方：

```
DATA: BEGIN OF gs_width,
      hkont TYPE i VALUE 30,
      xref1 TYPE i VALUE 12,
      kostl TYPE i VALUE 10,
      ...
END OF gs_width.

WRITE: AT /01(gs_width-hkont) ps_docit-hkont,
      AT      (gs_width-xref1) ps_docit-xref2, ...
```

步驟 3：宣告——型別、內表、命名 (講義 2、3、5、7)

- 用 `TYPES` 定義結構，內表用 `TYPE STANDARD TABLE OF`，另外宣告 work area，不用 **Header Line**。
- 命名照課程慣例：全域 `gt_ / gs_ / gv_`，FORM 內 `lt_ / ls_ / lv_`，FORM 參數 `pt_ / ps_ / pv_`。

- 選擇畫面的參考欄位用 `DATA gv_belnr TYPE bkpf-belnr.`，不用 `TABLES bkpf` (講義 7 §3.0)。
`TABLES` 只留畫面按鈕需要的 `sscrfields` (講義 28)。

```

TYPES: BEGIN OF ty_dochd,                                " 傳票表頭
        buhrs    TYPE bkpf-buhrs,
        belnr    TYPE bkpf-belnr,
        ...
        acctname TYPE zfi0037-acctname,                  " 會計主管姓名
END OF ty_dochd.

DATA: gt_dochd TYPE STANDARD TABLE OF ty_dochd,
      gs_dochd TYPE ty_dochd.                            " 目前輸出中的傳票 ( TOP-OF-PAGE 也讀它 )

```

步驟 4：選擇畫面與事件骨架 (講義 7、10、22、28)

主流程只剩 `PERFORM`，一眼看得出程式在做什麼：

```

AT SELECTION-SCREEN.
    PERFORM check_company_code.                            " 公司代碼存在嗎 ( MESSAGE E 擋在畫面 )
    PERFORM check_authority.                                " AUTHORITY-CHECK
    ...

START-OF-SELECTION.
    PERFORM build_status_range.                             " 三個勾選框 → NO-DISPLAY 的 s_bstat
    PERFORM get_header_data.                                 " 表頭 + 會計主管；查無資料 MESSAGE I +
STOP
    PERFORM get_item_data.                                   " 明細：過帳讀 BSEG，暫存讀 VBSEG*
    PERFORM get_text_data.                                  " 名稱類欄位

END-OF-SELECTION.
    PERFORM write_report.                                    " 一張傳票一個流程
    PERFORM fill_page_numbers.                              " 回填「頁碼 n/總頁數」

TOP-OF-PAGE.
    PERFORM write_header.
    PERFORM write_column_title.

```

三個勾選框轉成 `range` 表，讀表頭時只要一句 `AND bstat IN s_bstat` (講義 7 §3.2)：

```

LOOP AT gt_domval INTO ls_domval.                          " BSTAT 的 Domain 固定值
( GET_DOMAIN_VALUES )
    IF ( p_post = 'X' AND ls_domval-domvalue_1 <> 'V'
        AND ls_domval-domvalue_1 <> 'W'
        AND ls_domval-domvalue_1 <> 'Z' )
    OR ( p_park = 'X' AND ( ls_domval-domvalue_1 = 'V' OR ls_domval-domvalue_1 = 'W' ) )

```

```

OR ( p_pdel = 'X' AND ls_domval-domvalue_l = 'Z' ).
  ls_bstat-low = ls_domval-domvalue_l.
  APPEND ls_bstat TO s_bstat.
ENDIF.
ENDLOOP.

```

步驟 5：讀資料——整批讀，迴圈裡只查內表 (講義 6、11)

原則：資料庫一次讀完，迴圈裡只做 **READ TABLE**。

會計主管：一次讀出這個公司的所有主管，依生效日由新到舊排序，每張傳票在內表裡找第一筆「生效日不晚於傳票日期」的：

```

SELECT bukrs inauguration acctname INTO TABLE lt_acct
FROM zfi0037
WHERE bukrs = p_bukrs.
SORT lt_acct BY inauguration DESCENDING.

LOOP AT gt_dochd ASSIGNING <ls_dochd>.
  ... " 決定用哪一天 ( lv_date )
  LOOP AT lt_acct INTO ls_acct WHERE inauguration <= lv_date.
    <ls_dochd>-acctname = ls_acct-acctname. " 排最前面的就是當時的主管
  EXIT.
ENDLOOP.
ENDLOOP.

```

科目、成本中心、利潤中心、客戶 / 供應商名稱：先用 **FOR ALL ENTRIES** 整批讀進內表，再逐筆 **READ TABLE**。 **FOR ALL ENTRIES** 前面一定要空表防呆 (講義 11 §5)：

```

CHECK gt_docit IS NOT INITIAL. " 空表防呆

SELECT saknr txt50 INTO TABLE lt_skat
FROM skat
FOR ALL ENTRIES IN gt_docit
WHERE spras = sy-langu
  AND ktopl = gv_ktopl
  AND saknr = gt_docit-hkont.
...
LOOP AT gt_docit ASSIGNING <ls_docit>.
  READ TABLE lt_skat INTO ls_skat WITH KEY saknr = <ls_docit>-hkont.
  IF sy-subrc = 0.
    <ls_docit>-txt50 = ls_skat-txt50.
  ENDIF.
  ...
ENDLOOP.

```

每次查詢前先 **CLEAR** 目標變數，查到才採用（講義 4、ex19 的「殘留值」bug）。原程式查製票員姓名、客戶 / 供應商地址時沒有先清空，查不到就會沿用上一筆查到的值；範例程式的 `get_user_name` 一開始就 **CLEAR** `pv_name`，並把查過的帳號記在 `gt_username`，同一個人不重複查資料庫。

步驟 6：輸出——一張傳票一個流程（講義 8、12）

每張傳票：先數明細筆數，再逐筆輸出；每滿 10 筆或最後一筆，就印簽核欄與頁尾。

```
LOOP AT gt_dochd INTO gs_dochd.
  NEW-PAGE.                                " 每張傳票從新的一頁開始
  ...
  IF lv_item_doc = 'X' AND lv_item_cnt > 0.
    gv_item_mode = 'X'.                    " 讓 TOP-OF-PAGE 印欄位標題
    LOOP AT gt_docit INTO ls_docit WHERE bukrs = gs_dochd-bukrs
      AND belnr = gs_dochd-belnr
      AND gjahr = gs_dochd-gjahr.

      lv_idx = lv_idx + 1.
      PERFORM write_item USING ls_docit CHANGING lv_page_debit ...
      IF lv_idx MOD 10 = 0 OR lv_idx = lv_item_cnt.  " 每頁 10 筆，或最後一筆
        PERFORM write_sign_block USING lv_ppnam lv_usnam lv_page_debit ...
        CLEAR: lv_page_debit, lv_page_credit.
      ENDIF.
    ENDLOOP.
  ELSEIF ...                                " 沒有明細：只印表頭資訊與簽核欄
  ENDIF.
ENDLOOP.
```

- 這裡不用 **AT NEW**：`gt_dochd` 一列就是一張傳票，每一列都是新的一組，**AT NEW** 反而會把右邊欄位遮成 *（講義 20），還得再讀一次。
- 用「筆數到了沒」判斷最後一筆，比在 **LOOP ... WHERE** 裡用 **AT END OF** 清楚。
- 簽核欄有三種情況（沒有主管 / 暫存文件 / 已過帳），差別只在印誰、印哪一天，所以寫成一個 `write_sign_block`：先決定三個位置要印的姓名與日期，再用同一段 **WRITE** 印出來。同一段輸出不要複製三份（講義 8）。
- 中文內文依實際顯示寬度切成兩行，用 `cl_abap_list_utilities=>dynamic_output_length`（講義 12 §2.1）；標題置中也用它，不必轉編碼算位元組。

步驟 7：「頁碼 n/總頁數」（第 3.1 節）

每張傳票記下起訖頁，全部輸出完再回填；頁碼行的行號在印頁尾時用 `sy-linno` 記下來，不必寫死 64：

```
WRITE: AT /153(01) '|',
      AT 154 '頁    碼:',
      AT 164(20) gc_page_mark.            " 佔位符 '#PAGE#'
gv_page_line = sy-linno.                  " 記下頁碼在第幾行
```

不去改 `sy-pagno`，**READ LINE ... OF PAGE** 用的頁序就一直正確。

步驟 8：驗證——用真實資料和正式程式比對

範例程式寫好後，拿正式程式 ZRFI0004 當標準答案比對：比對程式 `cases/zr_tr13_zrfi0004_cmp.prog.abap` 用同樣的條件分別 `SUBMIT` 兩支程式 (`EXPORTING LIST TO MEMORY AND RETURN`)，用 `LIST_FROM_MEMORY`、`LIST_TO_ASCII` 把清單轉成文字，逐行比較。2026-09-29 實測，公司 1000：

條件	涵蓋的情況	輸出行數	結果
2026 年、過帳文件	剛好 10 筆明細的傳票、沒有明細的傳票	3,315	相同
2024 年、三種狀態全選	暫存文件、被刪除的暫存文件	8,905	相同
2022 年、文件 4900000000 ~ 040	12 ~ 18 筆明細 (跨 2 頁)	3,120	相同
2021 年、文件 5000000094 ~ 100	26 筆明細 (跨 3 頁)	650	相同

唯一的差異是**沒有明細的傳票的匯率**：原程式只在「有明細」的頁首把 0 匯率改成 1，沒有明細的傳票沒改到，而匯率欄位的 Domain 有轉換常式 `EXCRT`，會把 0 顯示成空白。於是同一支程式裡，有明細的傳票印 `1.00000`，沒有明細的印空白。範例程式統一印 `1.00000`。

驗證的心得：

- **邊界情況要刻意挑出來測**：剛好 10 筆 (換頁的邊界，原程式 2023/11/02 就修過這裡的頁數多算一頁)、跨多頁、沒有明細、暫存與被刪除的文件。
- **比對會找出原本沒人發現的問題**：這次找到匯率不一致、文件日期條件沒作用兩個。找到後要回報使用者確認，不是自己默默改掉。

自我檢查：這支程式好在哪裡

- `☐` 主流程只有 `PERFORM`，每個 `FORM` 做一件事，名稱看得出用途
- `☐` 命名照慣例，沒有 `Header Line`，沒有 `TABLES` 表工作區存資料
- `☐` 迴圈裡沒有 `SELECT ; FOR ALL ENTRIES` 前面有空表防呆
- `☐` 每個 `SELECT / READ TABLE` 後面都檢查 `sy-subrc`，查詢前先清空目標變數
- `☐` 版面寫在一個地方 (欄寬結構、頁碼行號用 `sy-linno` 記錄)，沒有到處寫死的數字
- `☐` 同一段輸出只寫一次 (簽核欄一個 `FORM`)
- `☐` 沒有改寫系統欄位
- `☐` 用真實資料驗證過，包含邊界情況

7. 課堂練習

完成 `ex13`：獨立完成航班營收報表。建議先不看答案程式，卡住時回查對應講義；完成後與 `zr_tr13_capstone.prog.abap` 對照，比較自己與範本的取舍差異。